Individual Project - Concise Guide

What to build, document, report, and submit

ASE

ASE @ Northern Kentucky University

1. Goal and Scope

Build a small Python application that solves a practical problem you care about. You work independently and own the plan, implementation, tests, documentation, weekly progress, and final submission.

Success means more than working code. Your repository should show that you can define a problem, make testable requirements, plan realistic milestones, improve a design, verify quality, and communicate progress.

1.1. Start Here

1. Choose a useful problem and identify the intended user.
2. Describe the proposed solution, features, and testable requirements.
3. Create one public GitHub repository with a clear README and organized code, tests, and documentation.
4. Create two sprints with milestones and weekly goals.
5. Link the repository and documents from your individual Canvas pages.

2. Required Work

- Build a prototype, then a minimum viable product (MVP).
- Improve the design through refactoring.
- Write and run all four required test types: unit, integration, regression, and acceptance. A missing test type is graded as an incomplete deliverable and is deducted accordingly.
- You may use AI to help write and run tests, but you must verify and understand every result yourself.
- Update your Canvas progress page every week.
- Prepare a user manual and an architecture/design document in Marp Markdown and PDF.
- Publish a portfolio page with Hugo on GitHub.io.
- Complete the individual presentation or progress checkpoint and/or the peer evaluation, depending on your course, as scheduled on Canvas.

2.1. Weekly Update

Each weekly update should answer:

1. What milestone or goal are you working toward?
2. Are you on track?
3. What did you complete this week?
4. What will you do next week?
5. What blockers or risks exist?
6. If you are behind, what is the revised plan and recovery date?
7. Where is the supporting GitHub evidence?

3. Timeline

- **Weeks 1-3:** problem, repository, plan, requirements, architecture, test outline, and weekly schedule
- **Week 4:** finish the individual planning materials required by HW2; there is no formal individual PPP unless Canvas says otherwise
- **Sprint 1 (from Week 5 until the first midterm):** working prototype/MVP
- **Right after the first midterm (about Week 9 or 10):** Sprint 1 progress submission required by HW4
- **Sprint 2 (after that, until Week 16):** features, design improvement, testing, documentation, and any Canvas-scheduled presentation/checkpoint or peer evaluation
- **Week 16:** final submission

Canvas is the official source for exact dates and presentation slots.

3.1. Shared Vocabulary and Team Ceremony Names

The team practice story uses **PPP (Project Plan Presentation), S1R (Sprint 1 Retrospective), S1P (Sprint 1 Presentation), S2R (Sprint 2 Retrospective), and FP (Final Presentation)**. It has no separate S2P. These are team ceremony names, not five additional individual presentations.

- **PPP:** team planning presentation in W4. Prepare your individual HW2 plan; do not assume a formal individual PPP is required.
- **S1R / S1P:** reflect on Sprint 1 before S1P (right after the first midterm); use the lessons to update the individual HW4 progress submission and Sprint 2 plan.
- **S2R / FP:** reflect before the W16 final review and submission; follow Canvas for any individual presentation slot or peer evaluation.
- **Retrospective:** process improvement, with concrete follow-up actions. **Presentation/review:** show working results and evidence to stakeholders.
- **Prototype:** feasibility experiment. **MVP:** Minimum Viable Product. **Definition of Done:** agreed implementation, tests, acceptance criteria, and documentation are complete.
- **Milestones** are checkpoints before the final **deadline**. Keep reporting problems and revised plans early; try dogfooding when your own application is useful to you.

The story places each retrospective shortly before its presentation. These are process examples, not fixed dates or permissions to hold class during a break.

4. Final Deliverables

- Public GitHub repository with `README.md`, source code, tests, and documentation
- Project plan, requirements, milestones, and weekly schedule
- Working application
- User manual: Marp source and exported PDF
- Architecture/design document: Marp source and exported PDF
- Weekly Canvas progress reports
- GitHub.io project page and Hugo source
- Final repository ZIP without generated files, unrelated files, or hidden `.git` data
- `ip_deliverables_checklist.md`
- `ip_self_eval_rubric.md`

5. Grading - 150 Points

Category	**Points**
Plan and weekly progress	50
Requirements and features	25
Implementation	25
Tests	25
Documentation and publication	25
Total	**150**

The point allocation is a guideline. The actual grading may differ by course, so always follow the current Canvas rubric. Use `ip_self_eval_rubric.md` for the detailed criteria. The instructor may adjust the self-evaluation after reviewing the submitted artifacts and progress evidence.

6. Rules and Final Check

- Do your own work and understand everything you submit.
- AI is allowed under the course AI policy. You must be able to explain every part and independently reproduce at least 80 percent of the essential code, reasoning, or steps without AI if asked later.
- Report problems early; do not hide a missed milestone.
- Follow the deadline and extension policy on Canvas. Send an extension request at least 48 hours before the deadline.
- Before submitting, open every link, run the application and tests, open each exported PDF, and verify the ZIP contents.